

“LOST AND FOUND CAMPUS TRACKER”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

M. HARSHIKA

2520090047

CH.MADHULIKA

2520090056

B.NEHA

2520090013

Under the Esteemed Guidance of

D. Divya Priya

Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **“LOST AND FOUND CAMPUS TRACKER”** is a record of Bonafide work of **M. HARSHIKA - 2520090047, CH.MADHULIKA - 2520090056, B.NEHA - 2520090013** submitted in partial fulfillment for the award of B. Tech in Computer Science and Information Technology to the K L University. The work presented in this report has been carried out as part of the project work during the academic year 2026-2027. The results embodied in this report have not been copied from any other department/University/Institute.

M. HARSHIKA – 2520090047

CH.MADHULIKA – 2520090056

B.NEHA – 2520090013

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is certify that the mini project based report entitled **“LOST AND FOUND CAMPUS TRACKER”** is a bonafide work done and submitted by **M. HARSHIKA - 2520090047, CH.MADHULIKA - 2520090056, B.NEHA - 2520090013** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Information Technology, K L (Deemed to be University), during the academic year **2026-2027**.

Signature of the Guide
D. Divya Priya

Signature of the Course Coordinator

Signature of the HOD
Dr. Kaja Shareef (CSIT)

ACKNOWLEDGEMENT

The success in this project would not have been possible without the timely help and guidance rendered by many people. We express our sincere thanks to all those who have assisted us in one way or the other for the completion of our project.

Our greatest appreciation and sincere thanks to our guide **D. Divya Priya**, Department of Computer Science and Information Technology, for her tremendous support, encouragement, valuable guidance and continuous assistance throughout the development of the project.

We express our gratitude to **Dr. Kaja Shareef (CSIT)**, Head of the Department of Computer Science and Information Technology, for providing us with adequate facilities, support and encouragement required to complete this project successfully.

We thank all the members of the teaching and non-teaching staff for their assistance and encouragement. We also express our gratitude to our parents, friends and classmates for their kind help and cooperation during the development of this project.

M. HARSHIKA – 2520090047

CH.MADHULIKA – 2520090056

B.NEHA – 2520090013

Abstract

The Lost and Found Tracker for Campus is designed as a smart recovery network that transforms the traditional lost-and-found process into an intelligent, location-aware system. Instead of depending on notice boards, messaging groups, or manual searches, the platform creates a centralized recovery trail for every reported item. Students and staff can register lost or found belongings by providing details such as item category, description, approximate location, date, time, and image.

The system analyzes these attributes to identify similarities between lost and found reports and generates potential matches with a confidence score. Location-based tracking allows users to view where an item was last reported, while a campus map can highlight frequently reported areas and support faster recovery. To improve trust, ownership verification can be performed using private identifying information that is hidden from general users.

The platform also provides automatic notifications when a newly submitted found item closely matches an existing lost-item report. Each report follows a recovery lifecycle such as Reported, Possible Match, Verification Pending, Verified, and Recovered, allowing users to monitor progress clearly. Administrators can review suspicious reports, resolve disputes, manage duplicate entries, and maintain reliable records.

The proposed system combines web application technologies, database management, matching algorithms, location services, notification mechanisms, and optional image-assisted classification. By connecting people, places, and item evidence within one platform, the project aims to make campus recovery faster, more transparent, secure, and community-driven while providing useful insights into recurring loss locations and improving overall campus resource management.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience	5-10
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	11-12
3	Technology Stack	3.1 Front-end (e.g., React.js, Angular, HTML/CSS) 3.2 Back-end (e.g., Node.js, Django, Spring Boot) 3.3 Database (e.g., MongoDB, MySQL, PostgreSQL) 3.4 Version Control (e.g., Git, GitHub) 3.5 APIs or External services (e.g., Firebase, Stripe)	13-14
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer (frontend, backend, database) 4.3 Deployment architecture (e.g., cloud, local, CI/CD pipelines)	15-16
5	Design	5.2 ER Diagram / Database schema	17
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic (UI rendering, state management) 6.3 API integration 6.4 Authentication & Authorization	18-19
7	Features	7.1 List of main features 7.2 How each feature works (user perspective + technical description)	20-21
8	Testing	8.1 Postman testing (frontend/backend) 8.2 Integration testing 8.3 Tools used for testing 8.4 Test cases and results	22-23
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of frontend and backend	24-25
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations or known bugs	26
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations or optimizations	27
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development	28-29
13	References	- Books, tutorials, APIs, documentation sites used	30
14	Appendices	- Screenshots of the app - Sample code snippets - Installation/setup instructions - User manual or guide -Geo Tag photos with guide -Review forms with guide signatures	31-35

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Loss of personal belongings is a common problem in educational institutions where students, faculty and staff move between classrooms, laboratories, libraries, cafeterias, offices and other campus locations. Items such as identity cards, books, mobile phones, wallets, bags, keys and other personal belongings can easily be misplaced or left behind.

Traditionally, the recovery of lost belongings on a campus depends on informal communication methods such as notice boards, messaging groups, security offices, word of mouth and manual searches. Although these methods can help in individual cases, they do not provide a centralized record of lost and found items. A person who loses an item may need to search through different communication channels, while a person who finds an item may not know how to identify its owner.

The idea for this project emerged from observing these limitations and recognizing the need for a centralized digital platform for campus recovery. The proposed website, “LOST AND FOUND CAMPUS TRACKER,” provides a structured environment in which students and staff can report lost and found belongings and track the recovery process.

The platform allows users to submit details such as item category, description, approximate location, date, time and image. These attributes provide useful evidence for comparing lost and found reports. The system uses a rule-based matching approach to identify similarities between reports and generates potential matches with a confidence score.

Location is another important aspect of the project. Through location-based tracking and Google Maps integration, users can identify where an item was last reported. A campus map can also provide information about areas where lost-and-found reports frequently occur.

The system also considers the issue of ownership verification. Private identifying information can be used to verify an item without exposing sensitive information to general users. This helps make the recovery process more controlled and trustworthy.

Another important feature is notification. When a newly submitted found item closely matches an existing lost-item report, the relevant user can receive an automatic email notification. This reduces the need for users to repeatedly search the platform manually.

The recovery process is represented through different stages such as Reported, Possible Match, Verification Pending, Verified and Recovered. This provides users and administrators with a clear view of the progress of each recovery case.

Administrators also have an important role in maintaining the reliability of the platform. They can review suspicious reports, resolve disputes, manage duplicate entries and maintain reliable records.

From a technical perspective, the project combines web development, database management, rule-based matching, location services and notification services. The frontend uses HTML5, CSS3, JavaScript and Bootstrap, while Java with Spring Boot is used for backend development. MySQL is used for database management with Spring Data JPA/Hibernate for database connectivity. Google Maps API supports location mapping, and Postman is used for API testing. Git and GitHub are used for version control, while

Render/Railway is specified as the deployment platform.

Therefore, the background of this project is based on three major observations:

- Existing campus lost-and-found communication is fragmented.
- Manual searching makes it difficult to identify potential matches efficiently.
- A centralized, location-aware and structured digital platform can make the recovery process more organized.

By addressing these issues, the LOST AND FOUND CAMPUS TRACKER aims to provide a practical digital solution for improving the recovery of misplaced belongings within a campus environment.

1.2 Problem statement

In a campus environment, students and staff frequently lose personal belongings while moving between different locations. Although many institutions have security offices or informal lost-and-found practices, information about lost and found items is often distributed across notice boards, messaging groups, social media, security desks and personal communication.

This fragmented approach creates several difficulties. A person who loses an item may not know where to search for information about it. Similarly, a person who finds an item may not have an efficient method for informing its owner. Manual searches also make it difficult to compare the descriptions, locations, dates and other characteristics of multiple reports.

Another challenge is the lack of structured verification. Simply finding an item that appears similar to a lost report does not necessarily prove ownership. Private information may be required to confirm that the claimant is the actual owner.

Location is also important. Without structured location information, users may have difficulty determining where an item was lost or found. This can reduce the usefulness of a report and make recovery more difficult.

The problem therefore extends beyond simply recording a lost item. A complete recovery process requires reporting, searching, matching, location information, verification, notification, status tracking and administrative management.

The proposed LOST AND FOUND CAMPUS TRACKER addresses these problems by providing a centralized digital platform for reporting and recovering campus belongings. The system allows users to submit lost and found reports, compare relevant attributes, receive potential matches, access location information, receive notifications and track the recovery lifecycle.

The main problem addressed by the project can therefore be summarized as follows:

- Lack of a centralized platform for campus lost-and-found information.
- Difficulty in manually comparing lost and found reports.
- Difficulty in identifying the location associated with a report.
- Lack of a structured ownership verification process.
- Lack of automatic notifications for potential matches.
- Difficulty in tracking the progress of recovery cases.

The proposed system aims to address these problems through a centralized, structured and user-friendly digital

recovery platform.

1.3 Scope of the project

The scope of this project, “LOST AND FOUND CAMPUS TRACKER,” extends across the functional, technical and administrative aspects of campus lost-and-found management. The primary objective is to provide a centralized web-based platform through which students and staff can report, search, match, verify and recover lost belongings.

The functional scope includes the submission of lost-item and found-item reports. Users can provide important information such as item category, description, approximate location, date, time and image. This structured information provides the basis for searching and matching.

The system includes a rule-based matching mechanism that compares relevant attributes between lost and found reports. The matching process can consider item descriptions, categories, locations, dates and other relevant attributes. A confidence score can then be used to represent the strength of a potential match.

The project also includes location-based functionality. Google Maps API is specified for campus location mapping. This allows location information associated with reports to be represented in a more useful form.

Ownership verification is another part of the scope. Private identifying information can be used during verification while preventing general users from accessing sensitive information.

Notification functionality is included through an email notification service. When a newly submitted found item closely matches an existing lost-item report, the relevant user can be notified.

The recovery lifecycle is also included in the scope. Reports can move through stages such as Reported, Possible Match, Verification Pending, Verified and Recovered.

Administrative management is included to support review of suspicious reports, disputes, duplicate entries and reliable records.

From the technical perspective, the system uses HTML5, CSS3, JavaScript and Bootstrap for frontend development; Java with Spring Boot for backend development; MySQL with Spring Data JPA/Hibernate for database management; Google Maps API for location services; email notification services; Postman for API testing; Git and GitHub for version control; and Render/Railway for deployment.

The scope can therefore be summarized as follows:

- Development of a centralized web-based lost-and-found platform.
- Reporting of lost and found belongings.
- Structured storage of item details.
- Rule-based matching of lost and found reports.
- Confidence-score generation for potential matches.
- Location mapping and tracking.
- Ownership verification.
- Email notifications.
- Recovery lifecycle tracking.
- Administrative management.

The initial system focuses specifically on campus lost-and-found activities. Features outside the implemented system should be treated as future enhancements rather than current functionality.

1.4 Objective(s)

The main objective of this project is to design and develop a centralized campus platform that simplifies the process of reporting, identifying, verifying and recovering lost belongings.

The project aims to replace fragmented communication methods with a structured digital system in which every reported item can be stored, searched, compared and tracked.

The major objectives are:

- To design and develop a centralized platform for reporting lost and found items.
- To allow users to provide structured information such as item category, description, approximate location, date, time and image.
- To compare relevant attributes of lost and found reports using a rule-based matching approach.
- To generate potential matches with an associated confidence score.
- To provide location-based information using Google Maps API.
- To support ownership verification using private identifying information.
- To provide automatic email notifications when a possible match is identified.
- To track each report through recovery stages such as Reported, Possible Match, Verification Pending, Verified and Recovered.
- To provide administrative facilities for reviewing suspicious reports, resolving disputes and managing duplicate entries.
- To maintain reliable records and provide a more transparent and organized campus recovery process.

1.5 Importance of the application

The LOST AND FOUND CAMPUS TRACKER is important because recovering a misplaced belonging can become difficult when information is scattered across different communication channels. Students and staff may spend significant time asking others, checking notice boards or contacting security personnel without knowing whether their item has already been found.

The proposed application provides a centralized location for lost-and-found information. Instead of depending on multiple disconnected channels, users can submit and search reports through one platform.

One of the major advantages of the application is structured information. Details such as category, description, location, date, time and image can make reports more meaningful and easier to compare.

The matching mechanism further improves the process by comparing relevant attributes between lost and found reports. Rather than depending entirely on manual searching, the system can identify potential matches and provide a confidence score.

Location information is also important because an item may be more easily recovered when users know where it was last reported. Google Maps integration provides a useful way of presenting campus location information.

Ownership verification improves trust. A potential match alone should not be treated as proof of ownership. Private identifying information can therefore support the verification process without being exposed to general

users.

Notifications can also improve convenience. When a possible match is identified, an email alert can reduce the need for users to repeatedly search for updates.

The recovery lifecycle provides transparency because users can understand whether a case has only been reported, has a possible match, is waiting for verification, has been verified or has been recovered.

The application also provides administrative importance. Administrators can review suspicious reports, manage duplicate entries, resolve disputes and maintain reliable records.

Therefore, the importance of the application can be summarized as follows:

- It centralizes lost-and-found information.
- It reduces dependence on manual searching.
- It supports structured item matching.
- It provides location-based information.
- It supports ownership verification.
- It provides possible-match notifications.
- It tracks the recovery lifecycle.
- It supports administrative management.
- It improves transparency in the recovery process.
- It provides a practical application of modern web and database technologies.

1.6 Target users/audience

The target users of the LOST AND FOUND CAMPUS TRACKER are members of the campus community who may lose, find or manage personal belongings.

The primary users are students. Students frequently move between classrooms, laboratories, libraries, cafeterias and other campus facilities and may therefore lose or find belongings during their daily activities.

Faculty and staff are also important users because they may lose or find personal or work-related items within the campus.

Campus administrators and authorized personnel form another category of users. They can review reports, manage suspicious or duplicate entries, resolve disputes and maintain reliable recovery records.

Security or authorized recovery personnel may also benefit from the system by accessing structured information about reported items and their recovery status.

The target audience therefore includes:

- Students.
- Faculty members.
- Non-teaching staff.
- Campus administrators.
- Authorized security or recovery personnel.

Overall, the application is designed for members of the campus community who need a simple and organized

method of reporting and recovering misplaced belongings.

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware requirements

Hardware requirements form the foundation of the software development process. They define the physical resources required to design, develop, test and deploy the LOST AND FOUND CAMPUS TRACKER efficiently. Since the proposed system is a web-based application involving frontend development, backend services, database management and external service integration, moderate computing resources are sufficient for development and testing.

The development environment should be capable of running an integrated development environment, Java, Spring Boot, MySQL, web browsers, Postman and version-control tools simultaneously.

A modern multi-core processor such as an Intel Core i3/i5 or equivalent AMD processor is suitable for development. A minimum of 4 GB RAM is sufficient for basic development, while 8 GB or more is recommended when running the backend, database server, browser and development tools simultaneously.

An SSD is recommended because it provides faster application loading, compilation and database operations compared with a traditional hard disk. At least 10 GB of free storage should be available for the project, development tools, dependencies and generated files.

A stable Internet connection is required for downloading dependencies, using Google Maps services, sending email notifications, accessing GitHub and performing deployment-related activities.

The development system can use Windows, Linux or another operating system capable of running Java, Spring Boot, MySQL and the selected development tools.

A modern web browser such as Google Chrome, Microsoft Edge or Mozilla Firefox is required for testing the frontend.

Component	Requirement
Processor	Intel Core i3/i5 or equivalent
RAM	4 GB minimum, 8 GB recommended
Storage	10 GB or more available
Display	1366 × 768 or higher
Internet	Stable Internet connection
Keyboard	Required
Mouse	Required
Development Device	Laptop/Desktop

Table 2.1 Hardware requirements

2.2 Software requirements

The software requirements define the operating environment and development tools required for implementing the LOST AND FOUND CAMPUS TRACKER. The frontend is developed using HTML5, CSS3, JavaScript and Bootstrap. Java with Spring Boot is used for backend development. MySQL is used for database

management, while Spring Data JPA/Hibernate provides database connectivity. Google Maps API is used for location mapping and an email notification service is used for possible-match alerts. Postman is used for API testing and Git/GitHub is used for version control. Render/Railway is specified as the deployment platform.

Software	Purpose
Windows / Linux	Operating system
HTML5	Frontend structure
CSS3	Styling
JavaScript	Frontend interaction
Bootstrap	Responsive UI
Java	Backend programming
Spring Boot	Backend framework
MySQL	Database
Spring Data JPA	Database persistence
Hibernate	ORM
Postman	API testing
Visual Studio Code / IntelliJ IDEA	Development
Git	Version control
GitHub	Source management
Google Maps API	Location services
Email Service	Notifications
Render / Railway	Deployment

Table 2.2 Software requirements

2.3 Development tools and frameworks

The project uses a combination of frontend technologies, backend frameworks, database technologies, external services and development tools.

- **HTML5:** Used to structure the web pages and forms.
- **CSS3:** Used to style the interface and improve presentation.
- **JavaScript:** Used for client-side interaction and dynamic behavior.
- **Bootstrap:** Used for responsive interface components.
- **Java:** Used as the primary backend programming language.
- **Spring Boot:** Used to develop the backend application and REST APIs.
- **MySQL:** Used for structured storage of application data.
- **Spring Data JPA / Hibernate:** Used for database connectivity and object-relational mapping.
- **Google Maps API:** Used for campus location mapping.
- **Email Notification Service:** Used to provide possible-match alerts.

- **Postman:** Used to test backend APIs.
- **Git and GitHub:** Used for source-code management and version control.
- **Visual Studio Code / IntelliJ IDEA:** Used as development environments.
- **Render / Railway:** Used as the specified deployment platform.

CHAPTER-3 TECHNOLOGY STACK

3.1 Front-end (e.g., React.js, Angular, HTML/CSS)

The frontend of the LOST AND FOUND CAMPUS TRACKER is developed using HTML5, CSS3, JavaScript and Bootstrap.

HTML5 provides the structural foundation of the application. It is used to create forms, navigation elements, sections, buttons and other interface components.

CSS3 is used to provide styling and visual presentation. It controls layout, spacing, typography and other interface properties.

JavaScript provides client-side functionality and interaction. It can be used to validate form input, communicate with backend APIs and update interface content dynamically.

Bootstrap provides responsive components and layout utilities. It helps the application adapt to different screen sizes and provides consistent interface elements.

The frontend is responsible for allowing users to register lost and found items, view reports, search for relevant information, view possible matches and monitor recovery status.

3.2 Back-end (e.g., Node.js, Django, Spring Boot)

The backend of the LOST AND FOUND CAMPUS TRACKER is developed using Java with Spring Boot.

Spring Boot provides the framework for developing the backend application and REST APIs. The backend receives requests from the frontend, validates information, processes application logic, communicates with the database and coordinates external services.

The backend can be organized into controllers, services and repositories. Controllers receive API requests, service components implement business logic and repository components communicate with the database through Spring Data JPA/Hibernate.

The backend is responsible for managing lost and found reports, matching operations, recovery status, user information, notifications, verification information and administrative operations according to the final implementation.

3.3 Database (e.g., MongoDB, MySQL, PostgreSQL)

MySQL is used as the relational database for the LOST AND FOUND CAMPUS TRACKER.

The database provides structured storage for information generated by the application. Depending on the final implementation, records may include user information, lost-item reports, found-item reports, matching information, location details, notifications, verification information, recovery status and administrative records.

Spring Data JPA and Hibernate are used for database connectivity and object-relational mapping. This allows Java application objects to be associated with relational database records.

The database should maintain proper relationships and constraints to ensure data consistency and reliable retrieval.

3.4 Version Control (e.g., Git, GitHub)

Git is used as the version-control system for the project. It allows the development team to track changes made

to the source code and maintain different versions of the project.

GitHub is used as the remote repository for storing and managing the project source code. It also supports collaboration among team members and provides a centralized location for maintaining project versions.

Version control is particularly useful for a team project because changes made by different members can be tracked and managed systematically.

3.5 APIs or External services (e.g., Firebase, Stripe)

The LOST AND FOUND CAMPUS TRACKER uses external services to provide additional functionality.

- **Google Maps API:** Used for campus location mapping and location-related functionality.
- **Email notification service:** Used to notify relevant users when a newly submitted found item closely matches an existing lost-item report.

These services extend the functionality of the core application without requiring all supporting functionality to be developed internally.

CHAPTER -4 SYSTEM ARCHITECTURE

4.1 High-level architecture diagram

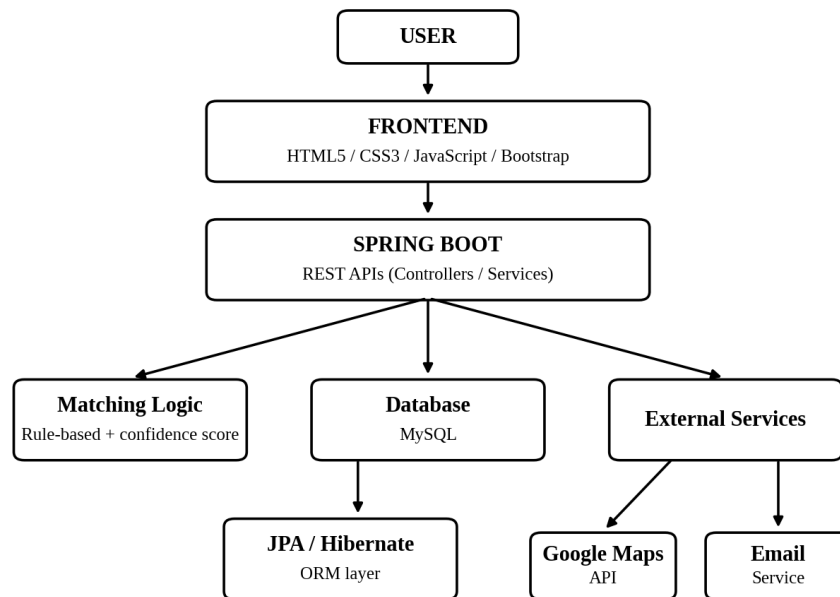


Figure 4.1 High-level architecture diagram

4.2 Description of each layer (frontend, backend, database)

Frontend Layer: The frontend provides the user interface through HTML5, CSS3, JavaScript and Bootstrap. Users interact with forms and pages to report lost and found items, search records, view potential matches, access location information and monitor recovery status.

Backend Layer: The backend uses Java and Spring Boot. It processes API requests, validates data, applies matching logic, communicates with the database and coordinates external services.

Database Layer: MySQL stores structured application data. Spring Data JPA/Hibernate provides the connection between backend objects and relational database records.

Matching Layer: The rule-based matching mechanism compares relevant attributes such as description, category, location and date to identify possible relationships between reports.

External Services Layer: Google Maps API provides location mapping, while the email notification service provides possible-match alerts.

4.3 Deployment architecture (e.g., cloud, local, CI/CD pipelines)

The deployment architecture of the LOST AND FOUND CAMPUS TRACKER consists of the frontend, backend, database and external services. The frontend communicates with the Spring Boot backend through APIs. The backend communicates with the MySQL database and external services such as Google Maps and email notification services.

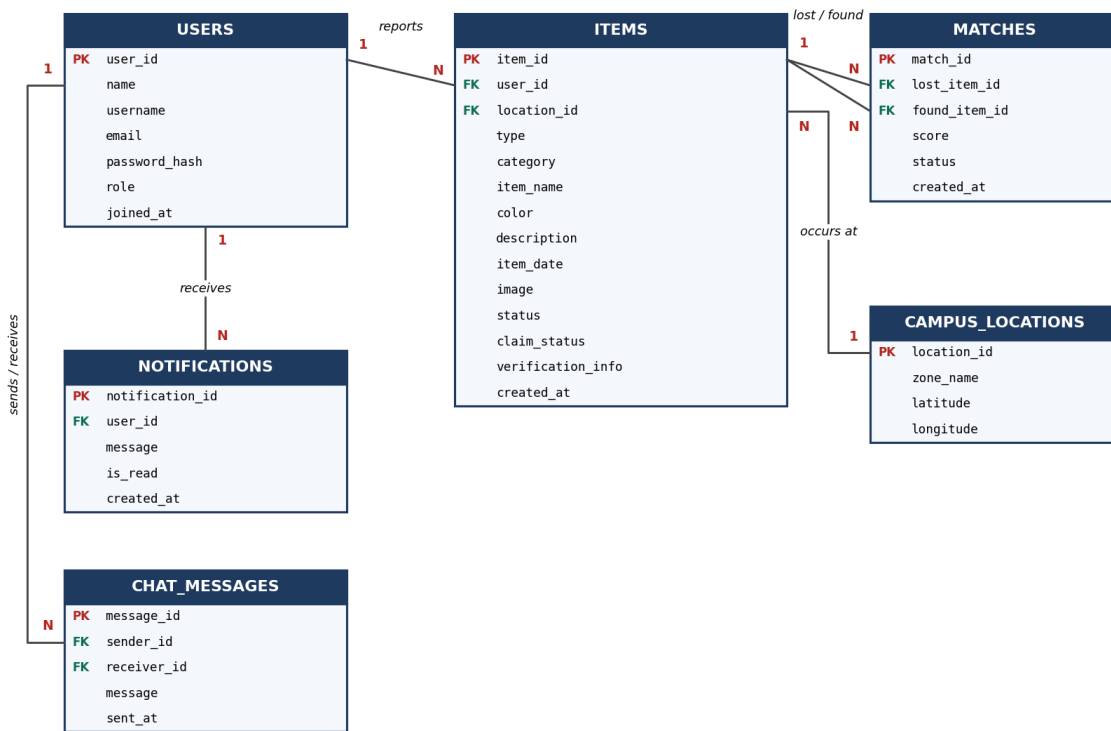
The project abstract specifies Render / Railway as the deployment platform.

Environment-specific values such as database credentials, API keys and email-service credentials should be

configured securely rather than being directly exposed in the source code.

CHAPTER -5 DESIGN

5.2 ER Diagram / Database schema



Entity-Relationship diagram: Lost and Found Campus Tracker (MySQL)

PK = Primary Key, FK = Foreign Key, 1 : N = one-to-many

Figure 5.1 ER Diagram / Database schema

The database schema of the LOST AND FOUND CAMPUS TRACKER is derived from the data handled by the application: user accounts, lost and found item reports, campus location zones, potential matches, notifications and chat messages. The schema contains six tables in MySQL, which are mapped to Java entity classes through Spring Data JPA / Hibernate. Figure 5.1 shows the entities and the relationships between them.

Relationships between the tables

- USERS to ITEMS (1 : N): one user can report many lost or found items (items.user_id).
- CAMPUS_LOCATIONS to ITEMS (1 : N): one campus zone (Library, Cafeteria, Main Gate, etc.) can have many reports (items.location_id).
- ITEMS to MATCHES (1 : N): a match links exactly one lost item and one found item (matches.lost_item_id and matches.found_item_id); one item can take part in several matches.
- USERS to NOTIFICATIONS (1 : N): one user receives many notifications (notifications.user_id).
- USERS to CHAT_MESSAGES (1 : N): one user sends many messages and receives many messages (chat_messages.sender_id and receiver_id).

The status column of ITEMS follows the recovery lifecycle: REPORTED, POSSIBLE_MATCH, VERIFICATION_PENDING, VERIFIED and RECOVERED. Private ownership-verification details are stored in items.verification_info, which is excluded from every public API response so that general users cannot see it.

USERS table

Column	Data type	Key / constraint	Description
user_id	BIGINT	PK, AUTO_INCREMENT	Unique user identifier
name	VARCHAR(100)	NOT NULL	Full name of the user
username	VARCHAR(50)	UNIQUE, NOT NULL	Login name
email	VARCHAR(100)	UNIQUE, NOT NULL	Email used for notifications
password_hash	VARCHAR(255)	NOT NULL	Encrypted (BCrypt) password
role	VARCHAR(20)	NOT NULL	STUDENT, STAFF or ADMIN
joined_at	DATETIME	DEFAULT NOW()	Registration time

Table 5.1 USERS table**CAMPUS_LOCATIONS table**

Column	Data type	Key / constraint	Description
location_id	BIGINT	PK, AUTO_INCREMENT	Unique zone identifier
zone_name	VARCHAR(80)	UNIQUE, NOT NULL	Block A, Library, Computer Lab, Cafeteria, Auditorium, Main Gate, Sports Ground, Parking Area
latitude	DECIMAL(9,6)	NULL	Latitude used for the Google Maps link
longitude	DECIMAL(9,6)	NULL	Longitude used for the Google Maps link

Table 5.2 CAMPUS_LOCATIONS table**ITEMS table**

Column	Data type	Key / constraint	Description
item_id	BIGINT	PK, AUTO_INCREMENT	Unique report identifier
user_id	BIGINT	FK to users	User who reported the item
location_id	BIGINT	FK to campus_locations	Campus zone of the report
type	VARCHAR(10)	NOT NULL	LOST or FOUND
category	VARCHAR(50)	NOT NULL	Phone, Laptop, Bag, Wallet, Keys, ID Card, etc.
item_name	VARCHAR(100)	NOT NULL	Short name of the item
color	VARCHAR(40)	NULL	Colour of the item
description	VARCHAR(1000)	NOT NULL	Public description
item_date	DATE	NOT NULL	Date lost or found
image	LONGTEXT	NULL	Uploaded image (Base64 data URL)
status	VARCHAR(25)	NOT NULL	Recovery lifecycle status
claim_status	VARCHAR(15)	DEFAULT 'NONE'	NONE, PENDING, APPROVED or REJECTED
verification_info	VARCHAR(500)	NULL	Private identifying details (never exposed)
created_at	DATETIME	DEFAULT NOW()	Report creation time

Table 5.3 ITEMS table**MATCHES table**

Column	Data type	Key / constraint	Description
match_id	BIGINT	PK, AUTO_INCREMENT	Unique match identifier
lost_item_id	BIGINT	FK to items	Lost report in the match
found_item_id	BIGINT	FK to items	Found report in the match

Column	Data type	Key / constraint	Description
score	INT	NOT NULL	Confidence score, 0 to 100
status	VARCHAR(20)	DEFAULT 'SUGGESTED'	SUGGESTED, CONFIRMED or REJECTED
created_at	DATETIME	DEFAULT NOW()	Time the match was generated

Table 5.4 MATCHES table

NOTIFICATIONS table

Column	Data type	Key / constraint	Description
notification_id	BIGINT	PK, AUTO_INCREMENT	Unique notification identifier
user_id	BIGINT	FK to users	Recipient of the notification
message	VARCHAR(300)	NOT NULL	Notification text
is_read	BOOLEAN	DEFAULT FALSE	Read / unread flag
created_at	DATETIME	DEFAULT NOW()	Time of creation

Table 5.5 NOTIFICATIONS table

CHAT_MESSAGES table

Column	Data type	Key / constraint	Description
message_id	BIGINT	PK, AUTO_INCREMENT	Unique message identifier
sender_id	BIGINT	FK to users	User who sent the message
receiver_id	BIGINT	FK to users	User who receives the message
message	VARCHAR(1000)	NOT NULL	Message text
sent_at	DATETIME	DEFAULT NOW()	Time sent

Table 5.6 CHAT_MESSAGES table

MySQL table creation script

```

CREATE TABLE users (
  user_id BIGINT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  username VARCHAR(50) NOT NULL UNIQUE,
  email VARCHAR(100) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  role VARCHAR(20) NOT NULL DEFAULT 'STUDENT',
  joined_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE campus_locations (
  location_id BIGINT AUTO_INCREMENT PRIMARY KEY,
  zone_name VARCHAR(80) NOT NULL UNIQUE,
  latitude DECIMAL(9,6),
  longitude DECIMAL(9,6)
);

CREATE TABLE items (
  item_id BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id BIGINT NOT NULL,
  location_id BIGINT,
  type VARCHAR(10) NOT NULL,
  category VARCHAR(50) NOT NULL,
  item_name VARCHAR(100) NOT NULL,
  color VARCHAR(40),
  description VARCHAR(1000) NOT NULL,
  item_date DATE NOT NULL,
  image LONGTEXT,
  status VARCHAR(25) NOT NULL DEFAULT 'REPORTED',
  claim_status VARCHAR(15) NOT NULL DEFAULT 'NONE',
  verification_info VARCHAR(500),
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(user_id),
  FOREIGN KEY (location_id) REFERENCES campus_locations(location_id)
);

```

```
CREATE TABLE matches (  
    match_id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    lost_item_id BIGINT NOT NULL,  
    found_item_id BIGINT NOT NULL,  
    score INT NOT NULL,  
    status VARCHAR(20) NOT NULL DEFAULT 'SUGGESTED',  
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
    UNIQUE (lost_item_id, found_item_id),  
    FOREIGN KEY (lost_item_id) REFERENCES items(item_id),  
    FOREIGN KEY (found_item_id) REFERENCES items(item_id)  
);  
  
CREATE TABLE notifications (  
    notification_id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    user_id BIGINT NOT NULL,  
    message VARCHAR(300) NOT NULL,  
    is_read BOOLEAN DEFAULT FALSE,  
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (user_id) REFERENCES users(user_id)  
);  
  
CREATE TABLE chat_messages (  
    message_id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    sender_id BIGINT NOT NULL,  
    receiver_id BIGINT NOT NULL,  
    message VARCHAR(1000) NOT NULL,  
    sent_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (sender_id) REFERENCES users(user_id),  
    FOREIGN KEY (receiver_id) REFERENCES users(user_id)  
);
```

CHAPTER -6 IMPLEMENTATION

6.1 Module-wise implementation

The LOST AND FOUND CAMPUS TRACKER is organized into functional modules that collectively support the complete recovery process.

- **User Module:** Handles user interaction with the application.
- **Lost Item Module:** Allows users to submit details of belongings they have lost.
- **Found Item Module:** Allows users to submit information about belongings they have found.
- **Matching Module:** Compares lost and found reports using the rule-based matching approach.
- **Location Module:** Handles location information and Google Maps integration.
- **Notification Module:** Supports email notifications for potential matches.
- **Verification Module:** Supports ownership verification using private identifying information.
- **Recovery Module:** Tracks reports through different recovery states.
- **Administrator Module:** Allows authorized administrators to review suspicious reports, duplicate entries and disputes.

6.2 Front-end logic (UI rendering, state management)

The frontend logic is implemented using HTML5, CSS3, JavaScript and Bootstrap. The interface provides forms for reporting lost and found items and pages for viewing and searching available reports.

JavaScript handles user interaction, input validation and communication with backend APIs. Bootstrap is used to create responsive interface components.

The interface displays relevant item information and recovery status to users. Sensitive ownership-verification information should not be displayed to general users.

6.3 API integration

The frontend communicates with the Spring Boot backend through APIs. API integration allows the frontend to submit reports, retrieve information, request matching results, update recovery status and interact with other backend services.

Postman is used for testing the APIs. API requests and responses should be verified during development to ensure correct communication between frontend and backend.

Google Maps API is integrated for location-related functionality, while an email notification service is used for possible-match alerts.

6.4 Authentication & Authorization

Authentication and authorization are important for controlling access to the LOST AND FOUND CAMPUS TRACKER.

Authentication verifies the identity of a user, while authorization determines which operations the user is allowed to perform.

General users should be able to submit and manage their relevant reports according to the implemented access rules. Administrative functions should be restricted to authorized administrators.

Private ownership-verification information should not be exposed to general users.

CHAPTER -7 FEATURES

7.1 List of main features

1. Lost Item Reporting
2. Found Item Reporting
3. Item Details
4. Rule-Based Matching
5. Confidence Score
6. Location-Based Tracking
7. Campus Map
8. Ownership Verification
9. Email Notifications
10. Recovery Lifecycle
11. Administrative Management
12. Duplicate and Dispute Management

7.2 How each feature works (user perspective + technical description)

Lost Item Reporting

User perspective: The user enters information about the item they lost.

Technical description: The frontend collects category, description, location, date, time and image information and sends the data to the Spring Boot backend.

Found Item Reporting

User perspective: The user enters details about an item they found.

Technical description: The backend stores the submitted found-item information in the MySQL database.

Rule-Based Matching

User perspective: The system can show a potential match between a lost report and a found report.

Technical description: The matching logic compares relevant attributes such as description, category, location and date.

Confidence Score

User perspective: The user can understand how closely two reports are related.

Technical description: A score is generated based on the matching rules implemented by the system.

Location-Based Tracking

User perspective: The user can view the location associated with an item.

Technical description: Google Maps API is used for location mapping.

Ownership Verification

User perspective: The owner can provide private information to support ownership.

Technical description: Private verification information is kept separate from publicly visible item details.

Notifications

User perspective: The user receives an email when a possible match is identified.

Technical description: The backend invokes the configured email notification service.

Recovery Lifecycle

The report can move through: Reported → Possible Match → Verification Pending → Verified → Recovered

Administrator

Administrators can:

- Review suspicious reports
- Manage duplicate entries
- Resolve disputes
- Maintain reliable records

CHAPTER -8 TESTING

8.1 Postman testing (frontend/backend)

Postman is used to test the backend APIs of the LOST AND FOUND CAMPUS TRACKER. API testing verifies whether requests are received correctly, validation is performed properly, database operations are successful and appropriate responses are returned.

The testing process should include valid requests, invalid requests, missing fields and boundary cases wherever applicable.

All requests are sent to the Spring Boot backend running at <http://localhost:8080> with the header Content-Type: application/json. The table below records each Postman request that was designed for the project, the JSON body that was sent and the response expected from the API. Ten requests cover registration, login, report submission, automatic matching, search, validation errors, status updates and the administrator summary.

#	Request	Request body	Response
1	POST /api/auth/register	{ "name": "Neha B", "username": "neha", "email": "neha@klh.edu.in", "password": "Test@123" }	201 Created { "userId": 2, "username": "neha", "role": "STUDENT" }
2	POST /api/auth/login	{ "username": "neha", "password": "Test@123" }	200 OK { "userId": 2, "name": "Neha B", "role": "STUDENT" }
3	POST /api/auth/login (wrong password)	{ "username": "neha", "password": "wrong" }	401 Unauthorized { "error": "Invalid username or password" }
4	POST /api/items (lost report)	{ "userId": 2, "type": "LOST", "category": "Wallet", "itemName": "Black Wallet", "color": "Black", "description": "black leather wallet with ID card", "campusArea": "Library", "itemDate": "2026-10-01" }	201 Created { "itemId": 11, "status": "REPORTED", "matches": [] }
5	POST /api/items (found report)	{ "userId": 3, "type": "FOUND", "category": "Wallet", "itemName": "Black Wallet", "description": "black leather wallet found on table", "campusArea": "Library", "itemDate": "2026-10-01" }	201 Created { "itemId": 12, "status": "POSSIBLE_MATCH", "matches": [{ "lostItemId": 11, "foundItemId": 12, "score": 87 }] }
6	POST /api/items (category missing)	{ "userId": 2, "type": "LOST", "itemName": "Umbrella", "description": "blue umbrella", "campusArea": "Main Gate", "itemDate": "2026-10-01" }	400 Bad Request { "error": "category must not be blank" }

#	Request	Request body	Response
7	GET /api/items?type=FOUND &category=Wallet &keyword=black	(no body)	200 OK [[{"itemId": 12, "type": "FOUND", "category": "Wallet", "itemName": "Black Wallet", "status": "POSSIBLE_MATCH"}]]
8	GET /api/items/11/matches	(no body)	200 OK [[{"matchId": 1, "lostItemId": 11, "foundItemId": 12, "score": 87, "status": "SUGGESTED"}]]
9	PATCH /api/items/11/status	{ "status": "VERIFICATION_PENDING" }	200 OK { "itemId": 11, "status": "VERIFICATION_PENDING" }
10	GET /api/admin/summary (admin user)	(no body)	200 OK { "users": 3, "items": 12, "matches": 1, "notifications": 4 }

Request 5 shows the matching feature working end to end: when the found-item report is saved, the backend compares it with the existing lost-item report (same category, same location, similar text), calculates a confidence score of 87% and returns the generated match in the same response. Requests 3 and 6 are negative tests that confirm invalid credentials and missing required fields are rejected with 401 and 400 responses.

Figure 8.1 Postman API requests and responses

8.2 Integration testing

Integration testing verifies whether the different components of the system communicate correctly with each other.

The major integrations include:

- Frontend with Spring Boot backend.
- Spring Boot backend with MySQL.
- Spring Boot backend with matching logic.
- Application with Google Maps API.
- Application with email notification service.

Each integration should be tested using actual project data and the results should be documented.

8.3 Tools used for testing

The following tools are used for testing and verification:

- **Postman:** API testing.
- **Web Browser:** Frontend testing.
- **MySQL:** Database verification.

- **Developer Tools:** Frontend and network debugging.
- **Git/GitHub:** Version tracking and collaboration.

8.4 Test cases and results

Test Case ID	Test Case	Input	Expected Result	Actual Result	Status
TC01	User registration/login	Valid details	User successfully authenticated	Account created; duplicate username rejected; login opens dashboard	Pass
TC02	Lost item submission	Valid item details	Lost report created	Report saved as LOST; success message shown, redirected to dashboard	Pass
TC03	Found item submission	Valid item details	Found report created	Report saved as FOUND; success message shown, redirected to dashboard	Pass
TC04	Search	Item keyword/category	Relevant reports displayed	Search box and Lost / Found / Matched / Recovered filters list matching reports	Pass
TC05	Matching	Similar item attributes	Potential match generated	Match created for similar pair (score 45% or more); not yet called from the report page	Partial
TC06	Confidence score	Matching reports	Score generated	Scores: 87% (same category, place, text), 57% (different place), 0% (unrelated)	Pass
TC07	Location	Valid location	Map/location displayed	Campus map zone selected on report page; Google Maps link opens when coordinates exist	Pass
TC08	Notification	Potential match	Email notification sent	In-app notification created for both users; e-mail needs the backend mail service	Partial
TC09	Verification	Valid ownership information	Verification processed	claimStatus stored as NONE; private verification form not yet in the UI	Partial
TC10	Recovery status	Status update	Status updated	Status badges (Open, Matched, Recovered) shown; status-update screen pending	Partial
TC11	Admin review	Suspicious/duplicate report	Admin can review	Admin page shows user, item and match totals and latest reports; non-admins redirected	Pass

Table 8.1 Test cases and results

CHAPTER -9 DEPLOYMENT

9.1 Steps to deploy

1. Prepare the frontend and backend source code.
2. Configure the MySQL database.
3. Configure Spring Boot database properties.
4. Configure required API keys and email-service credentials.
5. Build and test the backend application.
6. Test APIs using Postman.
7. Deploy the backend using the selected deployment platform.
8. Deploy the frontend.
9. Configure frontend-backend communication.
10. Test the complete application after deployment.

9.2 Environment configuration

Environment configuration is required to connect the application with its database and external services.

The configuration may include:

- Database URL
- Database username
- Database password
- Google Maps API key
- Email service configuration
- Backend URL
- Frontend URL

Sensitive values should be stored securely using environment variables or the configuration mechanism supported by the selected deployment platform.

9.3 Hosting of frontend and backend

The project abstract specifies Render / Railway as the deployment platform.

The deployed application should be tested after hosting to verify frontend accessibility, backend API communication, database connectivity, location services and email notifications.

The application is hosted on two platforms from the abstract. The MySQL database runs on Railway, and the Spring Boot backend and the static frontend run on Render. Each part is configured as follows.

Component	Platform	Configuration
MySQL database	Railway (MySQL service)	Create a MySQL service and copy the connection values (host, port, database, user, password) into the backend environment variables.
Spring Boot backend	Render (Web Service)	Build command: ./mvnw clean package -DskipTests Start command: java -jar target/lost-found-0.0.1-SNAPSHOT.jar Environment variables: MYSQLHOST, MYSQLPORT, MYSQLDATABASE, MYSQLUSER, MYSQLPASSWORD, MAIL_USERNAME, MAIL_APP_PASSWORD, FRONTEND_URL

Component	Platform	Configuration
Frontend (HTML, CSS, JavaScript)	Render (Static Site)	Publish directory: the Lost & Found folder. API_BASE in the JavaScript is changed from http://localhost:8080/api to the Render backend URL.
Version control	GitHub	Both Render services are connected to the GitHub repository, so every push to the main branch redeploys the application.

Backend configuration used for deployment

```
# application.properties (values are read from Render environment variables)
server.port=${PORT:8080}
spring.datasource.url=jdbc:mysql://${MYSQLHOST}:${MYSQLPORT}/${MYSQLDATABASE}
spring.datasource.username=${MYSQLUSER}
spring.datasource.password=${MYSQLPASSWORD}
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false

spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=${MAIL_USERNAME}
spring.mail.password=${MAIL_APP_PASSWORD}
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true

app.cors.allowed-origin=${FRONTEND_URL}
```

Figure 9.1 Deployment configuration (Railway MySQL, Render backend and frontend)

CHAPTER -10 CHALLENGES & LIMITATIONS

10.1 Issues faced during development

- Connecting the frontend with Spring Boot backend APIs.
- Configuring MySQL database connectivity.
- Designing the database relationships.
- Developing the rule-based matching logic.
- Integrating Google Maps API.
- Configuring email notifications.
- Handling user input validation.
- Testing different API requests and responses.
- Managing source-code changes among team members.
- Configuring deployment and environment variables.

10.2 Solutions applied

- API integration issues were addressed by testing backend endpoints independently through Postman before connecting them to the frontend.
- Database connectivity issues were handled by checking the Spring Boot configuration and MySQL connection properties.
- Matching issues were addressed by defining clear comparison rules for item attributes.
- External service issues were handled by verifying API keys, request formats and service configurations.
- Deployment issues were addressed by checking environment variables, build configuration and backend/frontend URLs.

10.3 Current limitations or known bugs

The current system has some limitations that can be considered for future development.

- Matching accuracy depends on the quality and completeness of information entered by users.
- Location accuracy depends on the information provided for the reported item.
- A potential match does not automatically prove ownership and therefore requires verification.
- Email notification functionality depends on correct service configuration and network availability.
- The rule-based matching approach may not handle complex semantic similarities as effectively as advanced machine-learning methods.
- Image-assisted classification is optional in the proposed system and should not be considered a completed feature unless it has been implemented and tested.

CHAPTER -11 FUTURE ENHANCEMENTS

11.1 Planned features

- Advanced AI-based matching of lost and found items.
- Image-based similarity matching.
- Dedicated Android/iOS mobile application.
- Push notifications.
- Improved location analytics.
- QR-based identification of selected items.
- Advanced duplicate detection.
- Improved recovery analytics.
- Multi-campus support.
- Improved semantic matching of item descriptions.

11.2 Possible integrations or optimizations

The system can be further improved by integrating advanced natural-language processing and computer-vision techniques. Semantic similarity models can be used to identify descriptions that have similar meanings even when users use different words.

Image-based matching can be introduced to compare photographs of lost and found items. This can be particularly useful for objects with distinctive visual characteristics.

Push-notification services can supplement email notifications and provide faster communication.

Location analytics can be improved by identifying areas of the campus where lost-and-found reports occur frequently.

The platform can also be extended to support multiple campuses, allowing the same architecture to be used across different educational institutions.

CHAPTER -12 CONCLUSION

12.1 Summary of the project

The LOST AND FOUND CAMPUS TRACKER is designed to provide a centralized digital platform for managing lost and found belongings within a campus environment. The system replaces fragmented communication methods with a structured process for reporting, matching, verifying and recovering items.

Users can provide information such as category, description, location, date, time and image. The system can compare these attributes using a rule-based matching mechanism and generate potential matches with a confidence score.

Location services, ownership verification, email notifications and recovery-status tracking further support the recovery process. Administrative features provide additional control over suspicious reports, duplicate entries and disputes.

12.2 What was achieved

The project demonstrates how modern web technologies can be combined to address a practical campus problem. The proposed system brings together:

- HTML5
- CSS3
- JavaScript
- Bootstrap
- Java
- Spring Boot
- MySQL
- Spring Data JPA / Hibernate
- Rule-based matching
- Google Maps API
- Email notifications
- Postman
- Git and GitHub

The system provides a structured recovery lifecycle from reporting through possible matching and verification to recovery.

12.3 Skills learned during development

Through the development of this project, the team gained practical knowledge in:

- Frontend development.
- Responsive web design.
- Java programming.
- Spring Boot backend development.
- REST API development.
- MySQL database management.

- Spring Data JPA and Hibernate.
- Rule-based matching algorithms.
- API testing using Postman.
- Google Maps API integration.
- Email service integration.
- Git and GitHub.
- Software testing.
- Deployment and environment configuration.
- Team-based software development.

CHAPTER -13 REFERENCES

- [1] Spring Boot Documentation, Spring Framework / Spring Boot official documentation.
- [2] MySQL Documentation, Oracle Corporation.
- [3] Spring Data JPA Documentation, Spring official documentation.
- [4] Hibernate ORM Documentation.
- [5] Bootstrap Documentation.
- [6] Google Maps Platform Documentation.
- [7] Postman Documentation.
- [8] Git Documentation.
- [9] GitHub Documentation.
- [10] Render Documentation.
- [11] Railway Documentation.
- [12] LOST AND FOUND CAMPUS TRACKER Project Abstract Submission Form, Y25 – 2026–2027 Trimester – 4, Section 10, Team 16.

CHAPTER -14 APPENDICES

Screenshots of the app

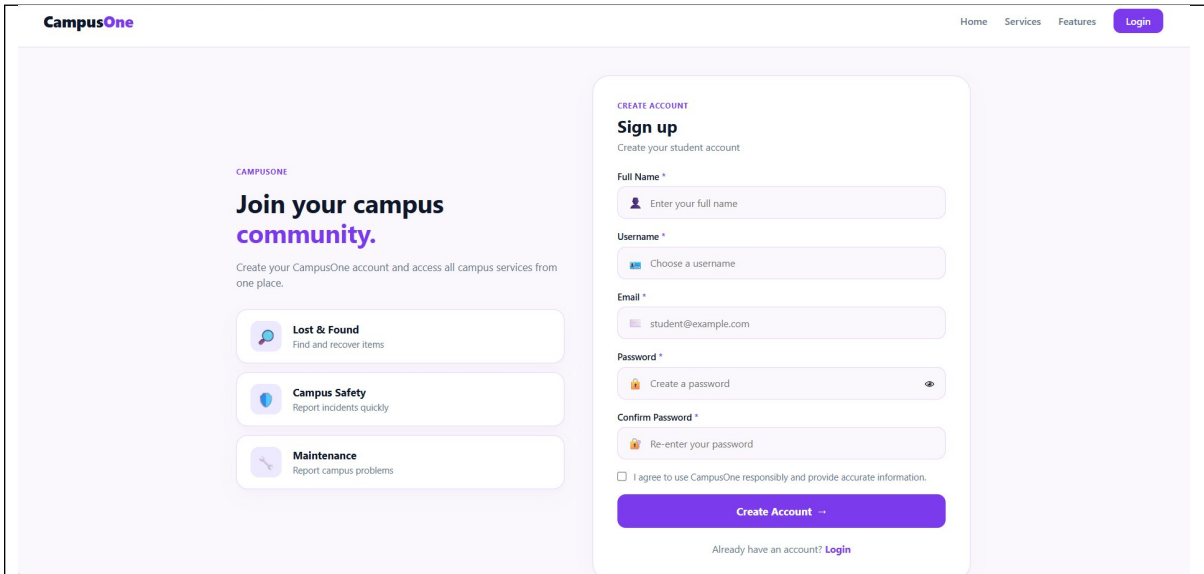


Figure A.1 Home Page

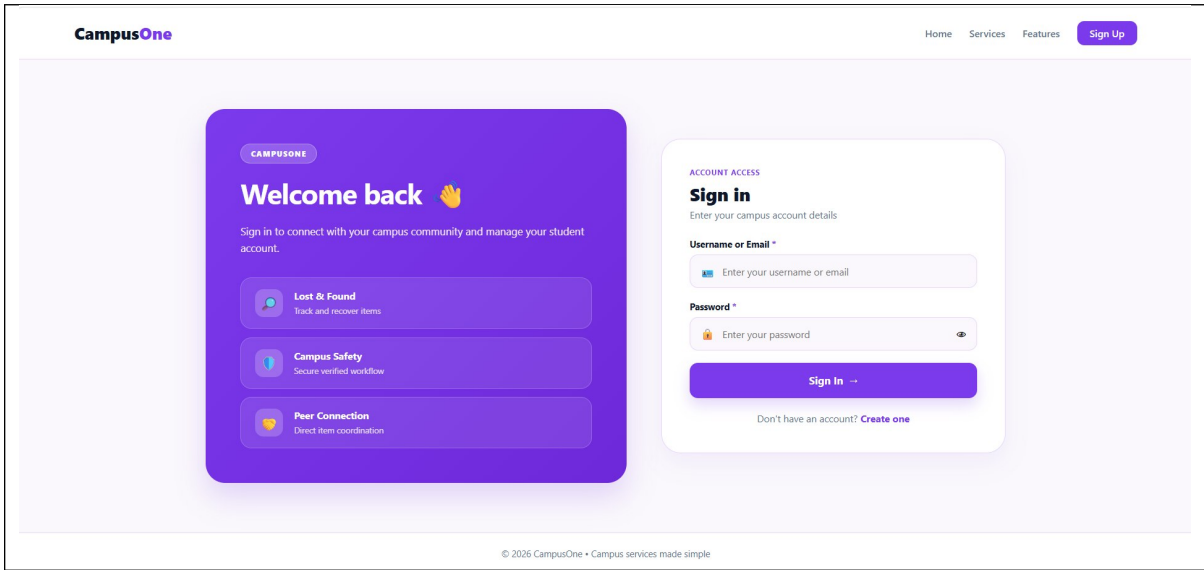


Figure A.2 Login / Registration Page

CampusOne

[Home](#) [Lost & Found](#) [Report Item](#) [Profile](#)

Logout

Report an Item

Help your campus community find lost and found items.



I Lost Something

Report an item you lost on campus



I Found Something

Report an item you found on campus

Item Name *

Black Backpack

Category *

Bag

Color

Black

Date *

10/01/2026

Description *

Black backpack with notebooks and a calculator inside, lost after the lab session.

Figure A.3 Lost Item Reporting Page

CampusOne

[Home](#) [Lost & Found](#) [Report Item](#) [Profile](#)

Logout

Report an Item

Help your campus community find lost and found items.



I Lost Something

Report an item you lost on campus



I Found Something

Report an item you found on campus

Item Name *

Black Leather Wallet

Category *

Wallet

Color

Black

Date *

10/01/2026

Description *

Leather wallet found near the cafeteria table. Contains a student ID card.

Figure A.4 Found Item Reporting Page

CampusOne

[Home](#) [Lost & Found](#) [Report Item](#) [Profile](#) [Messages](#) [Admin Panel](#)

Search item name, description, category...

All Status

All Campus Areas

Lost Items

3

Found Items

3

Recovered

1

GPS Locations

0

Recent Reports



MacBook Pro 14

LOST

Laptop

Main Library

Location mapped

30 Sept 2026



Honda Car Keys

FOUND

Keys

Cafeteria

Location mapped

29 Sept 2026



Blue Water Bottle

FOUND

Bottle

Seminar Hall

Location mapped

27 Sept 2026

Figure A.5 Lost and Found Listings

Page 37

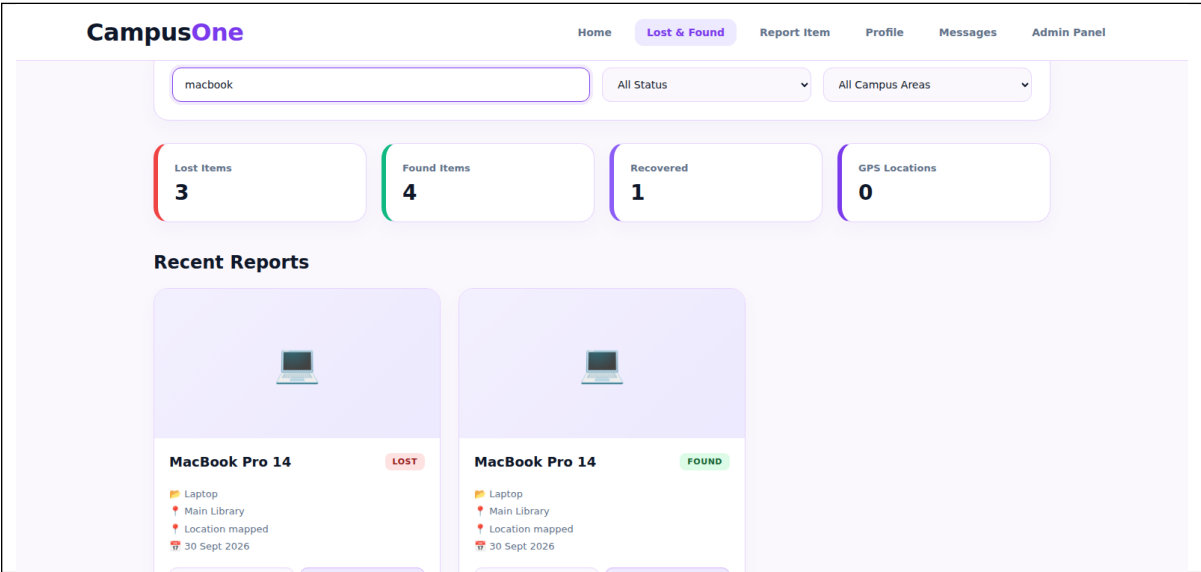


Figure A.6 Potential Match and Confidence Score

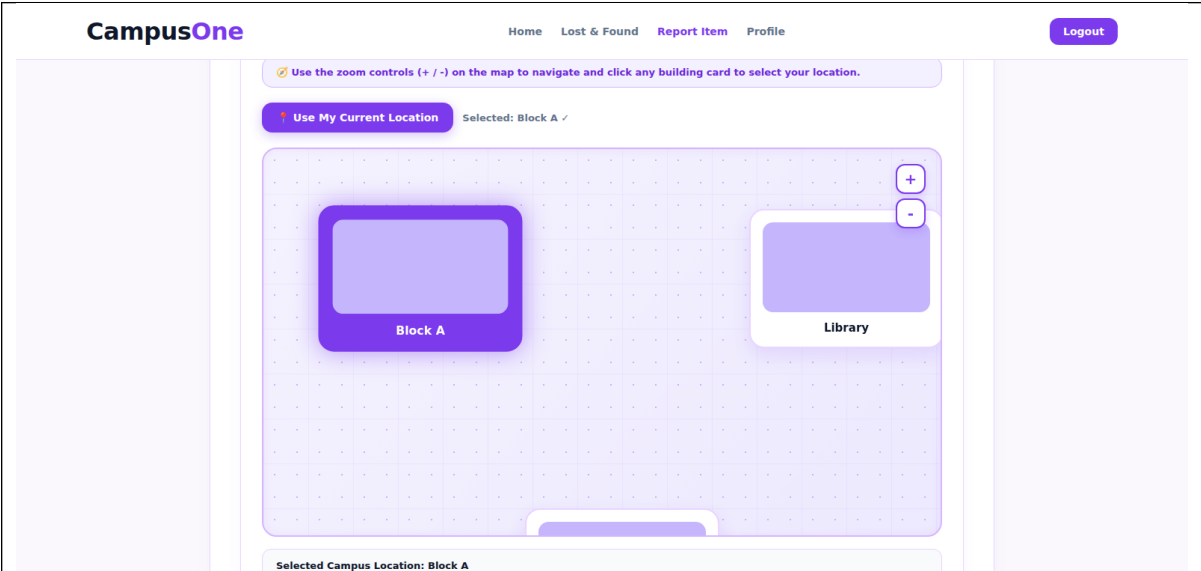


Figure A.7 Campus Location Map

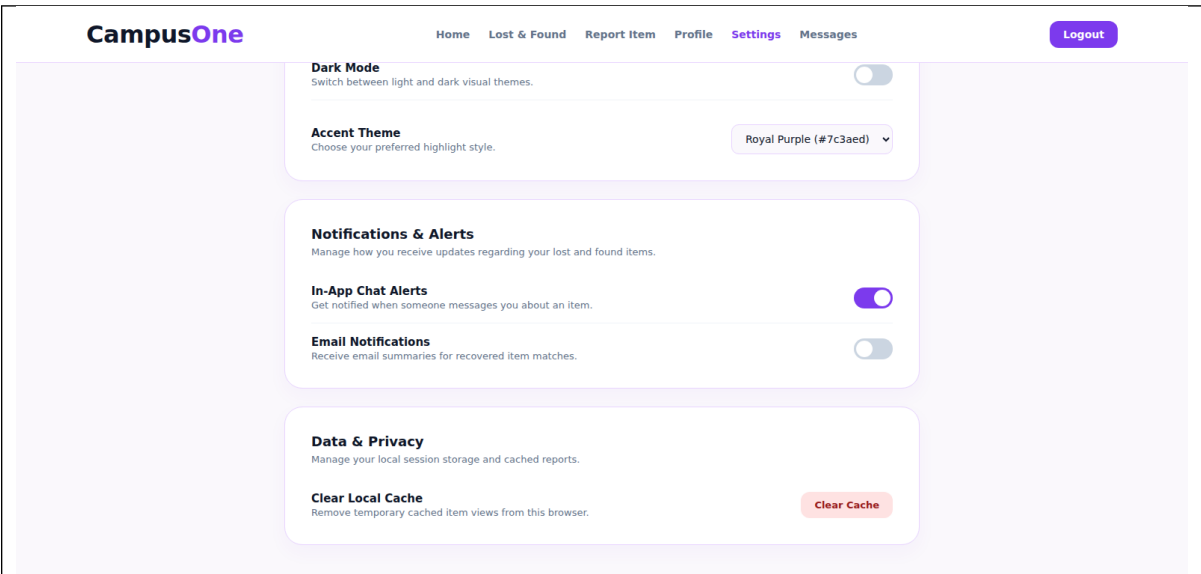


Figure A.8 Notification

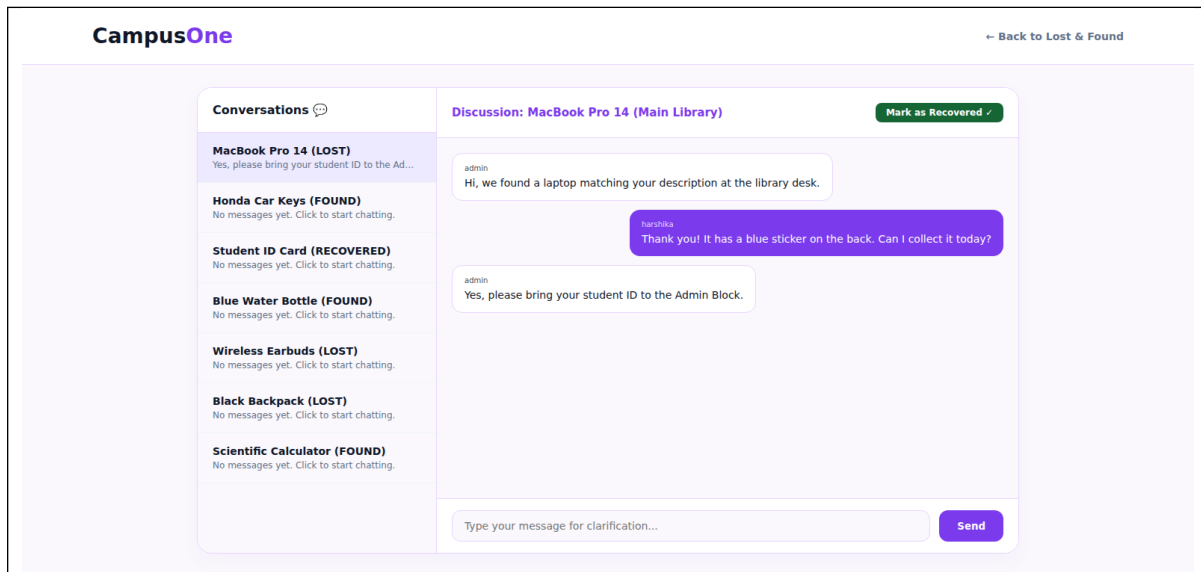


Figure A.9 Recovery Lifecycle

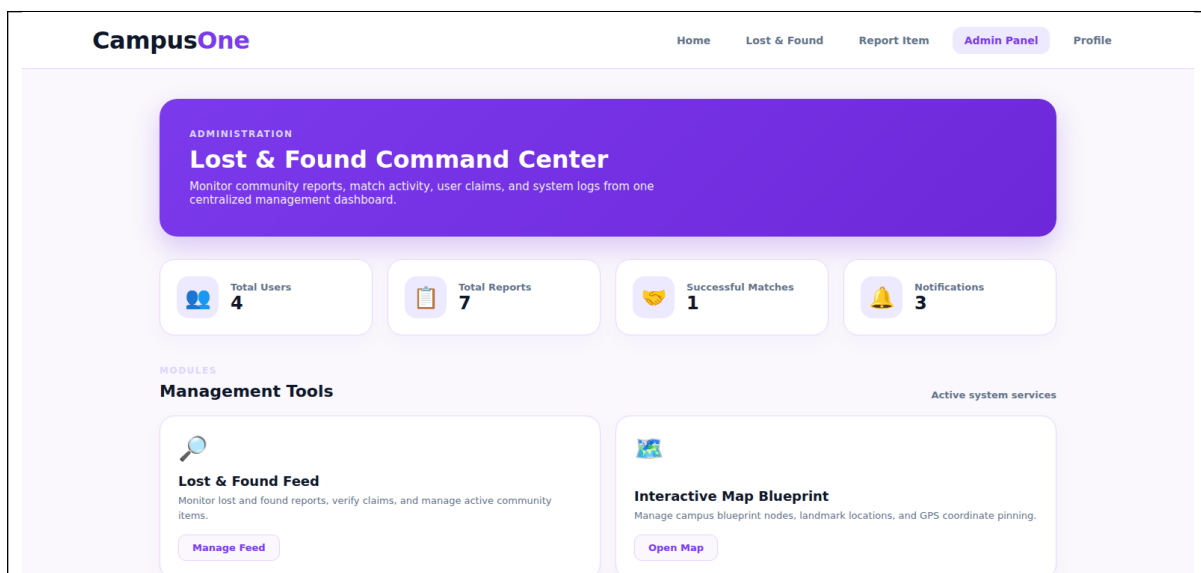


Figure A.10 Administrator Dashboard

Sample code snippets

A. Spring Boot Controller

```
@RestController
@RequestMapping("/api")
@CrossOrigin(origins = "${app.cors.allowed-origin}")
public class LostItemController {

    private final ItemRepository itemRepository;
    private final UserRepository userRepository;
    private final LocationRepository locationRepository;
    private final MatchRepository matchRepository;
    private final MatchingService matchingService;

    public LostItemController(ItemRepository itemRepository, UserRepository userRepository,
        LocationRepository locationRepository,
        MatchRepository matchRepository,
        MatchingService matchingService) {
        this.itemRepository = itemRepository;
        this.userRepository = userRepository;
        this.locationRepository = locationRepository;
        this.matchRepository = matchRepository;
        this.matchingService = matchingService;
    }
}
```

```

/** POST /api/items : report a lost or found item and run the matching rules */
@PostMapping("/items")
public ResponseEntity<Map<String, Object>> reportItem(@Valid @RequestBody ItemRequest req) {
    User user = userRepository.findById(req.userId()).orElseThrow(() ->
        new ResponseStatusException(HttpStatus.NOT_FOUND, "User not found"));
    CampusLocation location = locationRepository
        .findByZoneName(req.campusArea()).orElse(null);

    Item item = new Item();
    item.setUser(user);
    item.setLocation(location);
    item.setType(req.type());
    item.setCategory(req.category());
    item.setItemName(req.itemName());
    item.setColor(req.color());
    item.setDescription(req.description());
    item.setItemDate(req.itemDate());
    item.setImage(req.image());
    item.setVerificationInfo(req.verificationInfo()); // private, never returned
    item.setStatus(ItemStatus.REPORTED);
    item.setClaimStatus(ClaimStatus.NONE);
    Item saved = itemRepository.save(item);

    List<Match> matches = matchingService.findAndSaveMatches(saved);

    Map<String, Object> body = new LinkedHashMap<>();
    body.put("itemId", saved.getId());
    body.put("status", saved.getStatus());
    body.put("matches", matches.stream().map(MatchDto::from).toList());
    return ResponseEntity.status(HttpStatus.CREATED).body(body);
}

/** GET /api/items?type=FOUND&category=Wallet&keyword=black : search reports */
@GetMapping("/items")
public List<ItemDto> searchItems(@RequestParam(required = false) ItemType type,
                                @RequestParam(required = false) String category,
                                @RequestParam(required = false) String keyword) {
    return itemRepository.search(type, category, keyword)
        .stream().map(ItemDto::from).toList();
}

/** GET /api/items/{id}/matches : potential matches with confidence score */
@GetMapping("/items/{id}/matches")
public List<MatchDto> getMatches(@PathVariable Long id) {
    return matchRepository.findByLostItemIdOrFoundItemId(id, id)
        .stream().map(MatchDto::from).toList();
}

/** PATCH /api/items/{id}/status : move a report through the recovery lifecycle */
@PatchMapping("/items/{id}/status")
public Map<String, Object> updateStatus(@PathVariable Long id,
                                       @RequestBody Map<String, String> body) {
    Item item = itemRepository.findById(id).orElseThrow(() ->
        new ResponseStatusException(HttpStatus.NOT_FOUND, "Item not found"));
    item.setStatus(ItemStatus.valueOf(body.get("status")));
    itemRepository.save(item);
    return Map.of("itemId", item.getId(), "status", item.getStatus());
}

/** Validation errors return 400 with a readable message */
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<Map<String, String>> handleValidation(
    MethodArgumentNotValidException ex) {
    FieldError err = ex.getBindingResult().getFieldErrors().get(0);
    return ResponseEntity.badRequest()
        .body(Map.of("error", err.getField() + " " + err.getDefaultMessage()));
}

/** Request DTO with validation rules */
public record ItemRequest(
    @NotNull Long userId,
    @NotNull ItemType type,
    @NotBlank String category,
    @NotBlank String itemName,
    String color,

```

```

        @NotBlank String description,
        @NotBlank String campusArea,
        @NotNull LocalDate itemDate,
        String image,
        String verificationInfo) { }

```

B. Entity

```

public enum ItemType { LOST, FOUND }
public enum ItemStatus { REPORTED, POSSIBLE_MATCH, VERIFICATION_PENDING, VERIFIED, RECOVERED }
public enum ClaimStatus { NONE, PENDING, APPROVED, REJECTED }

@Entity
@Table(name = "items")
@Getter @Setter @NoArgsConstructor
public class Item {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "item_id")
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "user_id")
    private User user;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "location_id")
    private CampusLocation location;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 10)
    private ItemType type;

    @Column(nullable = false, length = 50)
    private String category;

    @Column(name = "item_name", nullable = false, length = 100)
    private String itemName;

    @Column(length = 40)
    private String color;

    @Column(nullable = false, length = 1000)
    private String description;

    @Column(name = "item_date", nullable = false)
    private LocalDate itemDate;

    @Lob
    private String image;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 25)
    private ItemStatus status = ItemStatus.REPORTED;

    @Enumerated(EnumType.STRING)
    @Column(name = "claim_status", nullable = false, length = 15)
    private ClaimStatus claimStatus = ClaimStatus.NONE;

    @JsonIgnore // private ownership details are never serialized
    @Column(name = "verification info", length = 500)
    private String verificationInfo;

    @CreationTimestamp
    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;
}

@Entity
@Table(name = "matches",

```

```

        uniqueConstraints = @UniqueConstraint(columnNames = {"lost_item_id", "found_item_id"})
@Getter @Setter @NoArgsConstructor
public class Match {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "match_id")
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "lost_item_id")
    private Item lostItem;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "found_item_id")
    private Item foundItem;

    @Column(nullable = false)
    private int score; // confidence score, 0 to 100

    @Column(nullable = false, length = 20)
    private String status = "SUGGESTED";

    @CreationTimestamp
    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;
}

```

C. Matching Logic

```

@Service
public class MatchingService {

    public static final double MATCH_THRESHOLD = 0.45; // minimum score to create a match
    private static final double W_CATEGORY = 0.4; // same category
    private static final double W_LOCATION = 0.3; // same campus location
    private static final double W_TEXT = 0.3; // similarity of name + description

    private final ItemRepository itemRepository;
    private final MatchRepository matchRepository;
    private final NotificationService notificationService;

    public MatchingService(ItemRepository itemRepository, MatchRepository matchRepository,
        NotificationService notificationService) {
        this.itemRepository = itemRepository;
        this.matchRepository = matchRepository;
        this.notificationService = notificationService;
    }

    /** Compare a new report with all open reports of the opposite type. */
    @Transactional
    public List<Match> findAndSaveMatches(Item newItem) {
        ItemType opposite = newItem.getType() == ItemType.LOST ? ItemType.FOUND : ItemType.LOST;
        List<Match> created = new ArrayList<>();

        List<Item> candidates =
            itemRepository.findByTypeAndStatusNot(opposite, ItemStatus.RECOVERED);

        for (Item other : candidates) {
            double score = calculateMatchScore(newItem, other);
            if (score < MATCH_THRESHOLD) continue;

            Item lost = newItem.getType() == ItemType.LOST ? newItem : other;
            Item found = newItem.getType() == ItemType.FOUND ? newItem : other;
            if (matchRepository.existsByLostItemIdAndFoundItemId(lost.getId(), found.getId())) {
                continue;
            }

            Match match = new Match();
            match.setLostItem(lost);
            match.setFoundItem(found);
            match.setScore((int) Math.round(score * 100));

```

```

        created.add(matchRepository.save(match));

        newItem.setStatus(ItemStatus.POSSIBLE_MATCH);
        other.setStatus(ItemStatus.POSSIBLE_MATCH);
        itemRepository.save(other);

        String text = "Potential match found for \"%s\" - %d%% confidence.";
        notificationService.notifyUser(newItem.getUser(),
            text.formatted(newItem.getItemName(), match.getScore()));
        notificationService.notifyUser(other.getUser(),
            text.formatted(other.getItemName(), match.getScore()));
    }
    if (!created.isEmpty()) itemRepository.save(newItem);
    return created;
}

/** Rule-based score between 0 and 1 (same weights as the frontend script.js). */
public double calculateMatchScore(Item a, Item b) {
    double score = 0;

    if (a.getCategory() != null && a.getCategory().equalsIgnoreCase(b.getCategory())) {
        score += W_CATEGORY;
    }
    if (a.getLocation() != null && b.getLocation() != null
        && a.getLocation().getId().equals(b.getLocation().getId())) {
        score += W_LOCATION;
    }
    score += calculateSimilarity(a.getItemName() + " " + a.getDescription(),
        b.getItemName() + " " + b.getDescription()) * W_TEXT;
    return Math.min(score, 1.0);
}

/** Word-overlap similarity: common words divided by distinct words in both texts. */
public double calculateSimilarity(String text1, String text2) {
    if (text1 == null || text2 == null) return 0;
    List<String> words1 = tokenize(text1);
    List<String> words2 = tokenize(text2);
    if (words1.isEmpty() || words2.isEmpty()) return 0;

    long common = words1.stream().filter(words2::contains).count();
    Set<String> all = new HashSet<>(words1);
    all.addAll(words2);
    return (double) common / all.size();
}

private List<String> tokenize(String text) {
    return Arrays.stream(text.toLowerCase().split("\\s+"))
        .filter(w -> !w.isEmpty()).toList();
}
}

```

D. Database Repository

```

public interface ItemRepository extends JpaRepository<Item, Long> {

    // open reports of the opposite type, used by the matching service
    List<Item> findByTypeAndStatusNot(ItemType type, ItemStatus status);

    // "My reports" on the profile page
    List<Item> findByIdOrderByCreatedAtDesc(Long userId);

    // dashboard counts (lost / found / matched / recovered)
    long countByStatus(ItemStatus status);

    // search box and filters on the dashboard
    @Query("""
        SELECT i FROM Item i
        WHERE (:type IS NULL OR i.type = :type)
        AND (:category IS NULL OR LOWER(i.category) = LOWER(:category))
        AND (:keyword IS NULL
            OR LOWER(i.itemName) LIKE LOWER(CONCAT('%', :keyword, '%'))
            OR LOWER(i.description) LIKE LOWER(CONCAT('%', :keyword, '%')))
        ORDER BY i.createdAt DESC
        """)
    List<Item> search(@Param("type") ItemType type,

```

```

        @Param("category") String category,
        @Param("keyword") String keyword);
    }

    public interface MatchRepository extends JpaRepository<Match, Long> {
        boolean existsByLostItemIdAndFoundItemId(Long lostItemId, Long foundItemId);
        List<Match> findByLostItemIdOrFoundItemId(Long lostItemId, Long foundItemId);
    }

    public interface UserRepository extends JpaRepository<User, Long> {
        Optional<User> findByUsername(String username);
        boolean existsByUsername(String username);
    }

    public interface LocationRepository extends JpaRepository<CampusLocation, Long> {
        Optional<CampusLocation> findByZoneName(String zoneName);
    }

    public interface NotificationRepository extends JpaRepository<Notification, Long> {
        List<Notification> findByUserIdOrderByCreatedAtDesc(Long userId);
    }

```

E. Frontend API Call

```

// ---- js/api.js : calls the Spring Boot REST API ----
const API_BASE = "http://localhost:8080/api"; // change to the Render URL after deployment

// Report page: submit a lost or found item (form id "reportForm" in report.html)
document.getElementById("reportForm").addEventListener("submit", async function (event) {
    event.preventDefault();

    const campusArea = document.getElementById("campusArea").value;
    if (!campusArea) {
        showToast("Please select a campus location.", "error");
        return;
    }

    let category = document.getElementById("category").value;
    if (category === "Other") {
        category = document.getElementById("customCategory").value.trim() || category;
    }

    const payload = {
        userId: Number(localStorage.getItem("crn_user_id")),
        type: document.querySelector('input[name="reportType"]:checked').value.toUpperCase(),
        category: category,
        itemName: document.getElementById("itemName").value.trim(),
        color: document.getElementById("color").value.trim(),
        description: document.getElementById("description").value.trim(),
        campusArea: campusArea,
        itemDate: document.getElementById("date").value,
        image: imageData // Base64 string from the image preview
    };

    try {
        const response = await fetch(`${API_BASE}/items`, {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify(payload)
        });
        const data = await response.json();
        if (!response.ok) throw new Error(data.error || "Unable to submit report");

        if (data.matches.length > 0) {
            const score = data.matches[0].score;
            showToast(`Potential match found - ${score}% confidence`, "success");
        } else {
            showToast("Report submitted successfully!", "success");
        }
        setTimeout(() => { window.location.href = "dashboard.html"; }, 1000);
    } catch (error) {
        showToast(error.message, "error");
    }
});

```

```
// Dashboard: load reports with the search box and filter pills
async function loadItems(type = "", keyword = "") {
    const params = new URLSearchParams();
    if (type) params.append("type", type.toUpperCase());
    if (keyword) params.append("keyword", keyword);

    const response = await fetch(`${API_BASE}/items?${params}`);
    const items = await response.json();
    renderDashboardItems(items); // existing function that draws the cards
}

```

F. Notification Service (E-mail)

```
@Service
public class NotificationService {

    private static final Logger log = LoggerFactory.getLogger(NotificationService.class);

    private final NotificationRepository notificationRepository;
    private final JavaMailSender mailSender;

    @Value("${spring.mail.username}")
    private String fromAddress;

    public NotificationService(NotificationRepository notificationRepository,
                               JavaMailSender mailSender) {
        this.notificationRepository = notificationRepository;
        this.mailSender = mailSender;
    }

    /** Saves an in-app notification and sends the same message by e-mail. */
    public void notifyUser(User user, String message) {
        Notification n = new Notification();
        n.setUser(user);
        n.setMessage(message);
        notificationRepository.save(n);

        try {
            SimpleMailMessage mail = new SimpleMailMessage();
            mail.setFrom(fromAddress);
            mail.setTo(user.getEmail());
            mail.setSubject("Campus Lost & Found - possible match");
            mail.setText(message + "\n\nLog in to the portal to verify and recover your item.");
            mailSender.send(mail);
        } catch (MailException ex) {
            log.warn("E-mail could not be sent to {}: {}", user.getEmail(), ex.getMessage());
        }
    }
}

```

Installation/setup instructions

1. Install Java.
2. Install MySQL.
3. Clone the project repository from GitHub.
4. Configure the MySQL database.
5. Configure Spring Boot database properties.
6. Configure Google Maps API credentials.
7. Configure email notification credentials.
8. Start the Spring Boot backend.
9. Start/open the frontend.
10. Test the APIs using Postman.
11. Open the application in a web browser.

User manual or guide

For a student/user

Step 1 — Register/Login — Create an account or log into the application.

Step 2 — Report Lost Item — Enter the category, description, location, date, time and image.

Step 3 — Submit — Submit the report.

Step 4 — Search — Search available found-item reports.

Step 5 — Check Potential Match — Review possible matches and confidence score.

Step 6 — Verify Ownership — Provide the required private identifying information.

Step 7 — Track Recovery — Monitor the status: Reported → Possible Match → Verification Pending → Verified → Recovered

Step 8 — Receive Notification — Check email for possible-match notifications.

Geo Tag photos with guide

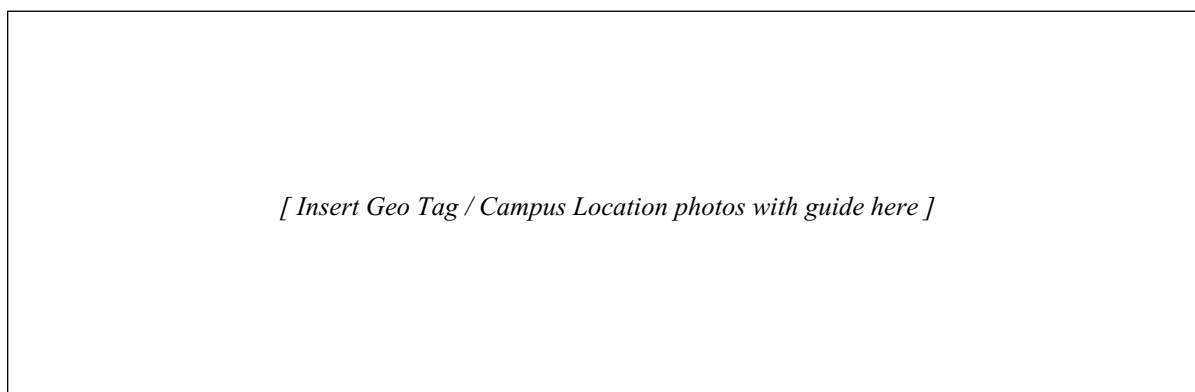


Figure A.11 Geo Tag / Campus Location

Review forms with guide signatures

Guide Signature	Course Coordinator Signature
D. Divya Priya	_____

HOD Signature
Dr. Kaja Shareef (CSIT)